DESIGN AND DEVELOPMENT OF AN INTERACTIVE UNIVERSITY COURSE REGISTRATION PORTAL

Saveetha Institute of Medical and Technical Sciences (SIMATS)
PROJECT REPORT

Name: Naveen P

Register Number: 192421020

Course: Web Technology (ITA0201)

1. Abstract

This project is an Interactive University Course Registration Portal built for SIMATS. It's a responsive, browser-based application where students can look through the available courses, fill in their registration details, pick the courses they want, and immediately see a summary of what they registered for. The page structure follows HTML/XHTML principles, CSS handles the presentation and responsive layout, and JavaScript takes care of client-side validation, course selection, credit calculation, and updating the page content.

The interface follows a glassmorphism style, using translucent panels, blur effects, rounded borders, hover states, and SIMATS branding, laid out with responsive Grid and Flexbox. Since this is an academic demonstration, the entire registration workflow runs on the client side, so the summary appears without the page needing to reload.

Course information is stored as a JavaScript array of objects. A single reusable function picks out the selected course objects and works out how many were chosen along with the total credits. The browser also checks that mandatory fields are filled in, the email format is valid, the semester is in range, and at least one course has been selected.

1.1 Keywords

HTML, XHTML, CSS, JavaScript, responsive web design, Flexbox, CSS Grid, glassmorphism, form validation, DOM manipulation, JavaScript objects, dynamic summary, client-side interaction.

1.2 Project Outcomes

- Build a structured university portal using semantic HTML elements.
- Present five academic courses in a tabular layout.
- Create a responsive and professional glassmorphism interface.
- Validate registration data before processing.
- Calculate selected course count and total credits dynamically.
- Display a registration summary without reloading the webpage.
- Use browser Developer Tools and console logging for debugging.

2. Introduction

Course registration is something every student goes through each semester, so a good registration interface matters more than it might seem. It should lay out course information clearly, cut down on input mistakes, and give feedback right away instead of leaving the student guessing. This project builds those requirements into a lightweight, client-side web application.

The portal splits its work across three technologies. HTML sets up the content hierarchy and form controls, CSS takes care of the visual presentation and how it adapts to different screen sizes, and JavaScript handles validation, rendering courses dynamically, doing the calculations, and general interactivity.

2.1 Problem Statement

A plain, static course listing can show information, but it can't validate what a student types in or work out a registration summary on its own. On top of that, a form without well-organized course information and responsive styling ends up being awkward to use. This project tries to fix both problems by bringing together a course catalogue, a registration form, validation logic, and a dynamic summary in one interface.

2.2 Objectives

- Build a semantic and well-organized university course registration page.
- Provide navigation links to Home, Courses, Registration, and Contact.
- Display at least five courses with code, name, credits, and type.
- Include registration instructions and eligibility requirements.
- Collect register number, student name, email, department, semester, and course choices.
- Apply responsive CSS using Grid/Flexbox and media queries.
- Validate mandatory fields, email, and semester values.
- Store course information as an array of JavaScript objects.
- Calculate selected course count and total credits through a reusable function.
- Display the registration summary dynamically without a page reload.

3. System Analysis and Requirements

3.1 Functional Requirements

Requirement	Implementation
University information	SIMATS branding, introductory text, contact details and navigation.
Course catalogue	Five courses presented in an HTML table.
Registration form	Register number, name, email, department, semester and multi-course checkboxes.
Validation	JavaScript checks required fields, email format, semester range and course selection.
Dynamic selection	Selected courses and credit values update live.
Credit calculation	Reusable calculateSelection() function.
Registration summary	Student and selected-course details displayed without a page reload.
Debugging	console.log() output used with browser Developer Tools.

3.2 Non-Functional Requirements

- Usability: straightforward for students to use.
- Responsiveness: usable across desktop, tablet and mobile screens.
- Maintainability: HTML, CSS and JavaScript are kept in separate files.
- Performance: calculations and summary updates occur locally in the browser.
- Accessibility: form labels are associated with controls and the summary uses an appropriate live region.

3.3 Technology Stack

Technology	Purpose
HTML5 / XHTML principles	Page structure, semantic elements, table and form markup.
CSS3	Glassmorphism design, box model, Grid, Flexbox and responsive media queries.
JavaScript	Validation, DOM manipulation, course data, calculations and event handling.
Google Chrome / Edge	Testing and browser Developer Tools.
VS Code or similar editor	Source-code development.

4. System Design

4.1 Architecture

The application follows a simple three-layer, client-side structure. HTML holds the page content and controls. CSS drives the glassmorphism look, layout, spacing, responsiveness, and visual states. JavaScript reads the form values, validates them, matches the checked boxes to the right course objects, calculates the totals, and updates the DOM. There's no backend or database involved — this was built purely for the assignment as a client-side demo.

4.2 Page Structure

Section	Main Content
Header	SIMATS branding and navigation.
Home	Portal introduction, call-to-action and registration visual.
Courses	Course catalogue table.
Instructions	Registration steps and eligibility list.
Registration	Student details and course selection.
Live Summary	Selected courses, total course count and total credits.
Contact/Footer	Institution contact information and footer.

4.3 Data Model

Course details are stored as an array of JavaScript objects, each one holding a code, name, credit value, and type. Both the course table and the selection controls are generated from this same array, so the data doesn't have to be written out twice.

Representative object: { code: 'WT201', name: 'Web Technology', credits: 4, type: 'Core' }

4.4 Process Flow

- The webpage loads and course details are rendered from the JavaScript array.
- The student enters registration details.
- The student selects one or more courses.
- JavaScript updates the live course and credit totals.
- The student submits the registration form.
- JavaScript validates all required values.
- If validation fails, an error message is displayed and submission is stopped.
- If validation succeeds, the registration summary is displayed without reloading the page.

5. User Interface and Front-End Design

5.1 Visual Design

The final interface goes with a glassmorphism-inspired look — dark blue/purple backgrounds paired with semi-transparent glass panels, backdrop blur, rounded corners, subtle borders, soft shadows, and light accent colors. SIMATS branding is worked into the header.

5.2 Navigation

A sticky header keeps the Home, Courses, Registration, and Contact links within reach at all times. Internal anchor links let users jump straight to different sections of the single-page portal.

5.3 Course Catalogue

Course Code	Course Name	Credits	Type
WT201	Web Technology	4	Core
DS202	Data Structures	4	Core
DB203	Database Management Systems	3	Core
CN204	Computer Networks	3	Core
AI205	Introduction to Artificial Intelligence	3	Elective

5.4 Responsive Design

CSS Grid handles the major layouts, like the hero and registration areas, while Flexbox takes care of navigation and lining up smaller components. On smaller screens, media queries collapse the multi-column sections down into a single column.

5.5 CSS Features Used

- Element, class, ID, pseudo-class and descendant selectors.
- Padding, margin, border, width, height and box-sizing for the box model.
- CSS Grid for major page layouts.
- Flexbox for navigation and component alignment.
- Hover and focus transitions on interactive elements.
- Sticky header and responsive media queries.
- Backdrop-filter blur and translucent backgrounds for the glassmorphism effect.

6. JavaScript Implementation

6.1 Course Data

The course catalogue lives in a JavaScript array called courses, where each entry is an object with the course code, name, credit value, and type. The table rows and the course-selection controls are both generated from this same array.

6.2 Validation

The validateForm() function checks the mandatory fields, whether the student name looks valid, the email format, the department selection, the semester range, and whether at least one course has been picked. If anything's wrong, the submission is blocked and an error message is shown so the student knows what to fix.

6.3 Dynamic Calculation

The calculateSelection() function reads through the checked course codes, looks up the matching objects in the courses array, counts how many courses were selected, and uses reduce() to add up the total credits. These values are then written straight to the live selection area.

For example, if a student selects Web Technology (4 credits), Data Structures (4 credits), and Computer Networks (3 credits), the summary shows 3 selected courses and 11 total credits.

6.4 DOM Interaction

Every time a checkbox is toggled, it triggers updateSelection(), which refreshes the selected course count and credit total on the page. On submit, event.preventDefault() stops the browser from reloading the page, and once validation passes, displaySummary() fills in the registration summary.

6.5 Debugging with Developer Tools

I used the browser's Developer Tools quite a bit while building this. console.log() statements print out successful validation results, the selected course data, and the total credits, which made it much easier to trace the data flow and catch issues along the way.

7. Testing and Results

7.1 Test Cases

Test Case	Input / Action	Expected Result
Empty form	Click Generate Registration Summary	Mandatory-field errors displayed.
Invalid email	student@	Email validation error.
Invalid semester	No selection / invalid value	Semester validation error.
No course	All course boxes unchecked	Course-selection error.
One course	Select one course	Course count = 1 and matching credits.
Multiple courses	Select 3 courses	Names and total credits update.
Valid registration	All valid fields + courses	Success summary shown.
Responsive view	Resize to mobile width	Columns stack and remain usable.
Debugging	Open F12 Console	Course and validation logs visible.

7.2 Sample Successful Result

Summary Field	Sample Value
Student Name	Sample Student
Register Number	24CSE101
Department	Computer Science and Engineering
Semester	Semester 5
Selected Courses	Web Technology; Data Structures; Computer Networks
Total Courses	3
Total Credits	11

7.3 Result Discussion

The finished portal covers the client-side requirements of the assignment. Course details show up in a table, the form collects all the required student information, and invalid inputs get caught before a summary is ever generated. Course selection and credit calculation both happen instantly in the browser, and the summary shows up without a full page refresh.

The responsive layout holds up well on smaller screens, and the browser Console gives clear evidence of the debugging that went into it. The glassmorphism styling ties everything together and gives the SIMATS portal a consistent visual identity.

8. Conclusion and Future Enhancements

8.1 Conclusion

This Interactive University Course Registration Portal shows how HTML, CSS, and JavaScript can come together to build a complete, responsive academic interface. HTML gives it a meaningful structure and working form controls, CSS delivers the polished glassmorphism layout, and JavaScript handles the validation, data handling, calculations, and dynamic updates to the page.

Overall, the project meets the assignment brief — it includes university information, navigation, a five-course table, lists and hyperlinks, a registration form, responsive CSS, browser-side validation, JavaScript course objects, a reusable calculation function, a dynamic registration summary, and debugging through Developer Tools.

8.2 Future Enhancements

- Connect the interface to a backend service and database so registrations persist.
- Add student authentication using institutional credentials.
- Prevent duplicate course registrations and enforce timetable-conflict checks.
- Add prerequisite checks and credit-limit validation.
- Generate a downloadable registration receipt or PDF.
- Provide an administrator dashboard for viewing and managing registrations.
- Add accessibility testing and support for multiple languages.

8.3 Project Folder Structure

File	Description
index.html	Semantic page structure, navigation, course table, form and summary.
style.css	Glassmorphism visual design, layout, responsive behavior and interactions.
script.js	Course data, validation, calculations, dynamic rendering and debugging.
images/	SIMATS/portal visual assets used by the interface.
README.md	Project execution notes and requirement mapping.

References

- HTML and CSS concepts drawn from Web Technologies Unit I.
- JavaScript concepts drawn from Web Technologies Unit II.
- Browser Developer Tools and the JavaScript Console, used for debugging.
- Official Saveetha / SIMATS website used as an institutional reference.

Appendix – Requirement Mapping

Question	Requirement	Evidence in Project
1	University title, navigation, courses, contact	SIMATS header, Home, Courses, Registration, Contact and Footer sections.
2	At least five courses	Five courses displayed in the HTML table.
3	Lists and hyperlinks	Registration instructions, eligibility list and internal navigation links.
4	Registration form	Register No., Name, Email, Department, Semester and Course Selection.
5	CSS and responsive interface	Selectors, box model, Grid, Flexbox, media queries, hover/focus effects and glassmorphism.
6	JavaScript validation	validateForm() checks required data, email, semester and course selection.
7	Course object/array	courses array contains course objects.
8	Reusable calculation	calculateSelection() returns selected courses, count and total credits.
9	Dynamic summary	DOM updates with student and selected-course information.
10	Developer Tools	console.log() used for validation, selection and registration debugging.